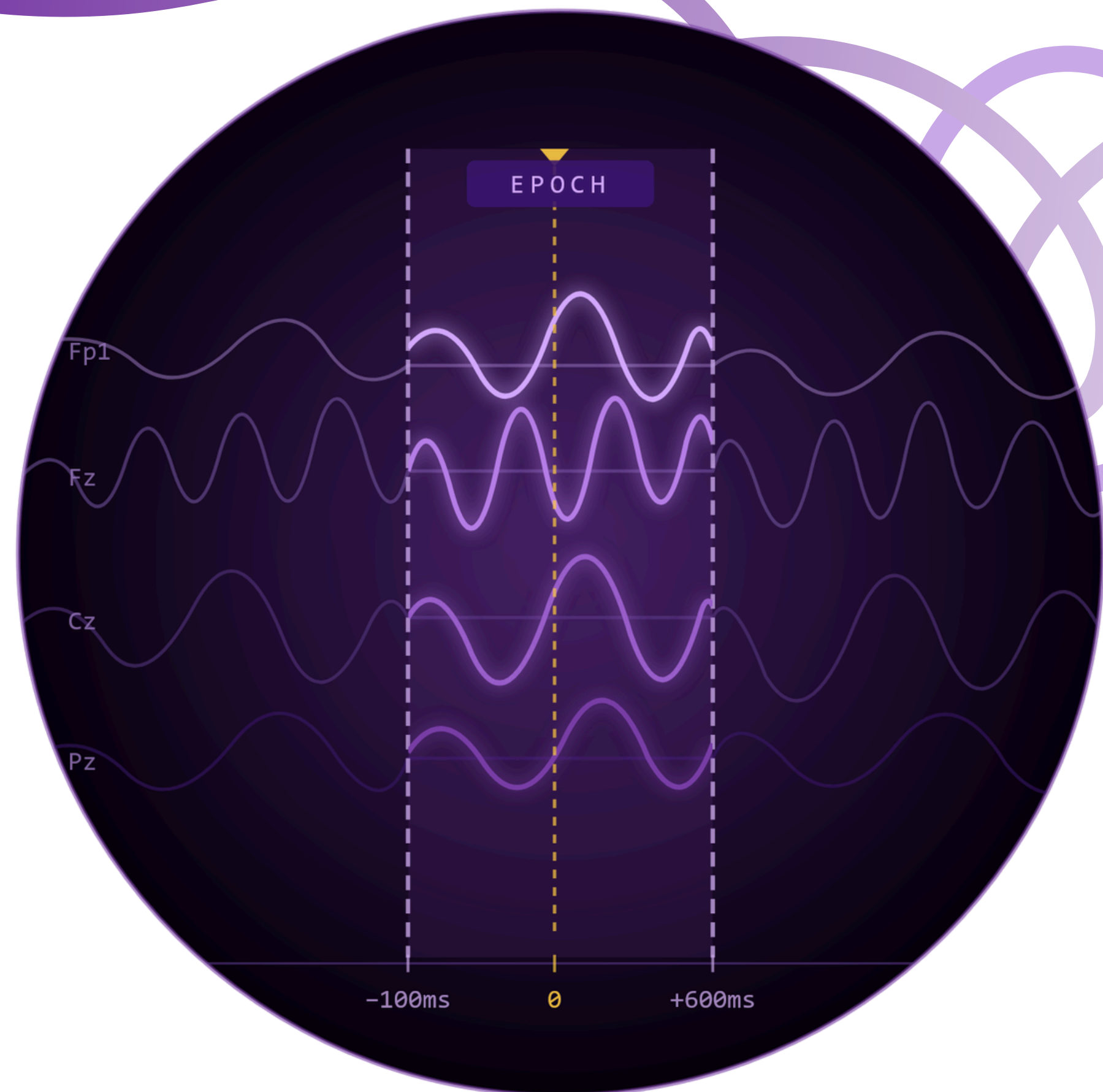


Epochs, Baseline & Artifact Correction



Contents

- Recap & Big Picture
 - Load, merge, filter, resample
- Re-referencing
 - Average reference, butterfly plot
- ICA – Artifact Correction
 - Fit on 1 Hz HP copy → apply on real data
- Extract Epochs
 - Cutting continuous EEG into event-locked segments
- Baseline Correction
 - Removing pre-stimulus drift
- Epoch Rejection
 - Threshold + for-loop over all participants

Recap –Day 1

1

Load & Merge

.vhdr/.vmrk/.eeg
concatenate_raws()
BAD boundary annotation

2

Filter

Band-pass 0.2–40 Hz
removes drift &
high-freq noise

3

Resample

256 Hz target pass
events = to avoid
timing drift

4

Save .fif

sub-XX_filtered_raw.fif
→ input for today

Key reminders

- BrainProducts files: .vhdr (header), .vmrk (markers), .eeg (binary) —always load the .vhdr
- `mne.concatenate_raws()` merges interrupted recordings — a 'BAD boundary' annotation marks the seam
- Always pass `events = to_resample()` so trigger timing stays aligned with the data after downsampling
- MNE best practice: high-pass filter on CONTINUOUS EEG before epoching to prevent edge artifacts near stimulus onsets (erpinfo.org)

Where Are We in the Pipeline?

Today: Steps 3–5 ● Go/No-Go Task ● Stimulus-locked epochs

Load &
Merge

Filter

Re-ref
& ICA

Extract
Epochs

Baseline
& Reject

Average

Why this order?

Filtering, re-referencing, and averaging are LINEAR operations — their order doesn't matter.

Artifact rejection is NONLINEAR (uses a threshold) — order does matter.

Best practice: filter → re-ref → ICA → epoch → reject. (Luck, 2014; erpinfo.org)

Paradigm: Go/No-Go Task

Participants pressed a button for 'Go' stimuli and withheld their response for 'No-Go' stimuli.

Epochs will be time-locked to stimulus onset, just like in the ERP slides you already saw.

We will compare Go vs. NoGo to examine N2 and P3 amplitude differences across conditions

Re-referencing

Why do we need it?

Every EEG signal is a voltage difference relative to a reference electrode. The online reference (set during recording) can bias the signal. We re-reference to the average of all channels - a spatially neutral signal.

How it works

Formula:

$$V_{\text{new}} = V_{\text{channel}} - \text{mean}(\text{all channels})$$

→ We apply this **BEFORE** ICA and epoching so both steps operate on unbiased data

```
raw_ref = raw.set_eeg_reference(ref_channels='average')  
raw_ref.apply_proj()
```

Plot before/after (butterfly plot)

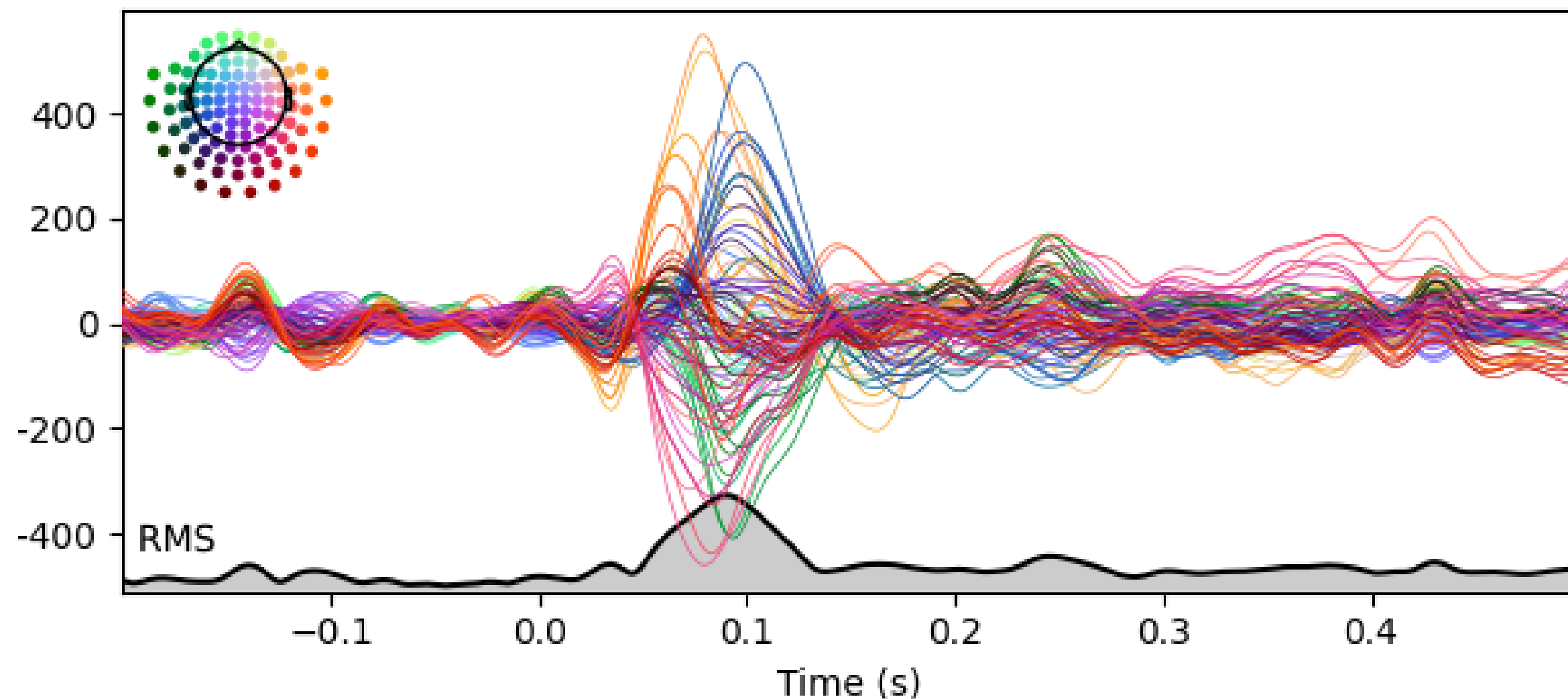
```
raw.copy().crop(100,102).plot(butterfly=True,  
                               title='Before')
```

```
raw_ref.copy().crop(100,102).plot(butterfly=True,  
                                   title='After')
```

→ what to look for in butterfly plots on next slide

What to look for in butterfly plots:

- Are all traces more centred around 0 μV after re-referencing?
- Is the overall amplitude spread smaller across channels?
- Any channel consistently far from the rest? $\rightarrow$ candidate for interpolation before continuing



ICA – Independent Component Analysis

1

Fit on 1 Hz HP copy

```
# Load ORIGINAL raw, apply 1 Hz HP  
# (needed: ICA needs drift-free data)
```

```
raw_ica = raw_orig.filter(  
    l_freq=1.0, h_freq=None)
```

```
ica = ICA(n_components=15,  
    max_iter='auto',  
    random_state=100)  
ica.fit(raw_ica)
```

2

Find Bad Components

```
# Topographic maps  
ica.plot_components()
```

```
# Time series  
ica.plot_sources(raw_ref)
```

```
# Auto-detect EOG  
eog_idx, scores = \  
    ica.find_bads_eog(raw_ref)  
ica.plot_scores(scores)
```

3

Preview & apply

```
# Before/after overlay  
ica.plot_overlay(raw_ref,  
    exclude=bad_components)
```

```
# Apply to real data (not raw_ica!)  
raw_clean = ica.apply(  
    raw_ref.copy(),  
    exclude=bad_components)
```

ICA decomposes the EEG signal into **statistically independent components**. Artifacts (blinks, heartbeat) tend to dominate certain components. We identify and remove them, then reconstruct the clean signal.

Artifacts – What Are They?

Ocular

- Blinks: large spikes at Fp1/Fp2, symmetric frontal topography, power <5 Hz
- Saccades: step-function time series, left/right asymmetric frontal map
- → ICA blink component is almost always the most obvious one

Cardiac

- Clear QRS complex in time series at ~1 Hz
- Nearly linear gradient scalp topography (heart far from head)
- No spectral peaks — rhythmic pulses across temporal channel

Muscular

- Broadband high-frequency power (20 Hz+) in power spectrum
- Shallow, focal scalp topography (not inside skull)
- Non-stationary bursts — coughing, jaw clenching, head movement

Technical

- Single-channel focal topography (one electrode popped or loose)
- 1/f power spectrum — unlike muscle components
- 50/60 Hz sharp peak = line noise → notch filter or exclude channel

Brain components: dipolar scalp map + 1/f power spectrum with a peak between 5–30 Hz (often 10 Hz alpha) + low dipole residual variance (<15%) — KEEP these!

What Is an Epoch?

An epoch is a **fixed-length segment** of EEG data, cut from the continuous recording, time-locked to a specific **event of interest**.

Instead of analysing hours of raw signal, we focus only on what happens around each stimulus — enabling ERP analysis (as covered in the EEG intro!).

Key Parameters

tmin	Start relative to event (e.g. -0.1 s)
tmax	End relative to event (e.g. 0.6 s)
event_id	Which events to epoch (Go / NoGo)
baseline	Interval for correction (next slide!)
reject	Threshold for auto-rejecting bad epochs

Extracting Epochs with MNE

```
# Find events from annotations in the raw file
events, event_id = mne.events_from_annotations(raw)
event_id_select = {'Go': 1, 'NoGo': 2}

# Extract epochs: -100 ms to 600 ms relative to event onset
epochs = mne.Epochs(
    raw_clean,          # ICA-cleaned, re-referenced data
    events, event_id=event_id_select,
    tmin=-0.1, tmax=0.6,
    baseline=None,      # apply separately below
    preload=True
)
print(epochs)
print(f'Go: {len(epochs["Go"])}, NoGo: {len(epochs["NoGo"])}')
```

Baseline Correction

Why do we need it?

EEG signals have slow drifts and DC offsets that vary across trials. Baseline correction removes pre-stimulus activity so that post-stimulus changes reflect genuine neural responses.

How it works

1. Define baseline window (e.g. -100 to 0 ms)
2. Compute mean amplitude in that window
3. Subtract that mean from the entire epoch

→ At $t = 0$, all channels start at $\sim 0 \mu V$
→ Filtering continuous EEG first prevents filter edge artifacts from contaminating the baseline (erpinfo.org)

MNE code

```
epochs.apply_baseline(baseline=(-0.1, 0))
```

- Why -100 to 0 ms? It is before the stimulus → represents the brain's resting state just before the event

```
# Or set when creating epochs:  
mne.Epochs(..., baseline=(-0.1, 0))
```

```
# Verify: mean in window  $\approx 0 \mu V$   
check = epochs.copy().crop(-0.1, 0)  
print(check.get_data().mean() * 1e6) # expect  $\sim 0$ 
```

If >20% of your baseline is noisy, consider a wider pre-stimulus window or check for bad channels first

ICA Is Not Enough?

Even after ICA, some epochs may still contain residual artifacts — muscle bursts, electrode pops, anything ICA did not isolate into its own component.

We use a threshold to automatically discard them. Artifact rejection is nonlinear —always do it LAST

Threshold Rejection

- Discard any epoch where any EEG channel exceeds $\pm 150 \mu\text{V}$. Simple and fast.
- If one channel causes most rejections: interpolate that channel and rerun. If $>20\%$ rejected $\rightarrow$ threshold too strict or bad channels need attention first

```
reject = {'eeg': 200e-6} n_before = len(epochs) # 150
# 150  $\mu\text{V}$  in Volts
epochs.drop_bad(reject=reject) n_after = len(epochs)
pct = (n_before - n_after) / n_before * 100
```

```
# See what was dropped:
epochs.plot_drop_log() # which channels?
```

Manual / Visual Inspection

- Plot epochs and click to mark bad ones
- interactively. More accurate, more effort. Useful for final QC on key channels or when automatic threshold is insufficient.

```
epochs.plot(
    n_epochs=10,
    n_channels=20,
    scalings='auto'
)
```

Click epochs/channels to mark bad

Your Task Today

Morning Practice (Notebook)

1. Load sub-01 data (already filtered)
2. Apply average re-referencing
3. Fit ICA on 1 Hz HP copy
4. Inspect: plot_components, plot_sources
5. Apply ICA to re-referenced data
6. Extract epochs (Go & NoGo)
7. Baseline correction (-100 to 0 ms)
8. Reject epochs above 150 μ V

Final Task: All Participants

Take your single-participant notebook and wrap everything in a for-loop over all participants. Save the cleaned epochs per participant as a .fif file.

```
for sub_id in participants:
    raw = load_data(sub_id)
    raw_ref = re_reference(raw)
    ica.fit(raw_1hz)
    raw_clean = ica.apply(raw_ref)
    epochs = extract_epochs(raw_clean)
    epochs.apply_baseline(...)
    epochs.drop_bad(reject=...)
    epochs.save(f'{sub_id}-epo.fif')
```


Key Takeaways

Re-reference to average before ICA and epoching — removes spatial bias

ICA: fit on 1 Hz HP copy of ORIGINAL raw, apply on Day 1 filtered data

Epochs cut continuous EEG into event-locked windows (just like ERP trials)

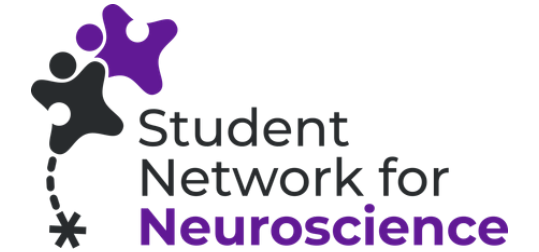
Baseline correction removes pre-stimulus drift — all epochs start at $\sim 0 \mu V$

Threshold rejection discards epochs still too noisy after ICA

Can you figure that out yourself?

Always keep in mind to look at MNEs official documentation!
<https://mne.tools/stable/documentation/index.html>

Sources



- Gramfort, A. et al. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267. <https://doi.org/10.3389/fnins.2013.00267>
- MNE-Python Documentation – Preprocessing & ICA: https://mne.tools/stable/auto_tutorials/preprocessing/index.html
- MNE-Python ICA Tutorial: https://mne.tools/stable/auto_tutorials/preprocessing/40_artifact_correction_ica.html
- AGhaderi/MNE-Preprocessing (GitHub): <https://github.com/AGhaderi/MNE-Preprocessing>
- Luck, S. J. (2014). *An Introduction to the Event-Related Potential Technique* (2nd ed.). Bradford Books.
- Ghaderi-Kangavari, A. et al. (2024). An easy-to-follow handbook for EEG data analysis with Python. ResearchGate. <https://doi.org/10.13140/RG.2.2.22208.51207>
- Zschocke, S., & Hansen, H. (2023). *Klinische Elektroenzephalographie* (4th ed.). Springer. <https://doi.org/10.1007/978-3-662-63267-3>
- Bokura, H., Yamaguchi, S., & Kobayashi, S. (2001). Electrophysiological correlates for response inhibition in a Go/NoGo task. *Clinical Neurophysiology*, 112(12), 2224–2232.
- Harper, J., Malone, S. M., & Bernat, E. M. (2014). Theta and delta band activity explain N2 and P3 ERP component activity in a go/no-go task. *Clinical Neurophysiology*, 125(1), 124–132.
- Wessel, J. R. (2018). Prepotent motor activity and inhibitory control demands in different variants of the go/no-go paradigm. *Psychophysiology*, 55(3), e12871.